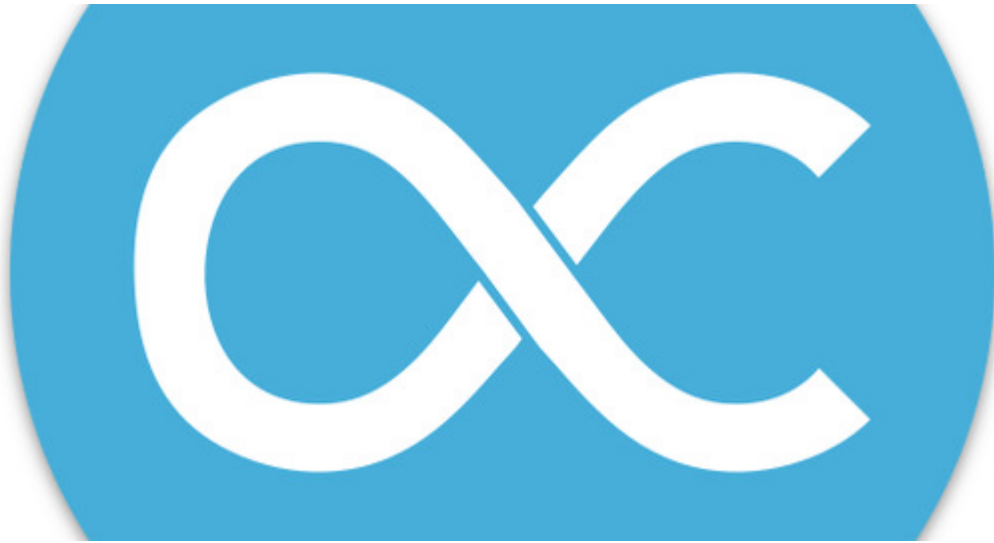


Proxying two connections in Go

 www.stavros.io/posts/proxying-two-connections-go/



I have recently taken an interest in the Go programming language (or golang, thanks, unsearchable name), and I thought that a fun weekend project would be to write a Postgres connection pooler that didn't require separate authentication, and just passed through to the database the credentials it got from the client.

Mid-way through the implementation, I realized that this wouldn't work due to the way Postgres does authentication, but I had written the pooling already. To do the proxying of sockets back and forth, I wrote two small snippets that might be interesting to someone.

The first snippet takes a `net.Conn` instance and returns a channel that anything the connection sends will be pushed to:

```

// chanFromConn creates a channel from a Conn object, and sends everything it
// Read()s from the socket to the channel.
func chanFromConn(conn net.Conn) chan []byte {
    c := make(chan []byte)

    go func() {
        b := make([]byte, 1024)

        for {
            n, err := conn.Read(b)
            if n > 0 {
                res := make([]byte, n)
                // Copy the buffer so it doesn't get changed while read by the
recipient.
                copy(res, b[:n])
                c <- res
            }
            if err != nil {
                c <- nil
                break
            }
        }
    }()

    return c
}

```

I found most of this code in [a blog post about Reading to channels](#), and it came with comments by Rob Pike et al on the [go-nuts](#) mailing list, so the code seems solid (and worked fine for me).

The second snippet, the meat of the proxy, takes two connections and pipes one to the other:

```
// Pipe creates a full-duplex pipe between the two sockets and transfers data from
one to the other.
func Pipe(conn1 net.Conn, conn2 net.Conn) {
    chan1 := chanFromConn(conn1)
    chan2 := chanFromConn(conn2)

    for {
        select {
        case b1 := <-chan1:
            if b1 == nil {
                return
            } else {
                conn2.Write(b1)
            }
        case b2 := <-chan2:
            if b2 == nil {
                return
            } else {
                conn1.Write(b2)
            }
        }
    }
}
```

As you can see, all it does is wait on the two channels for something to come in, and then send it to the other channel. It's pretty simple, but it might save someone (or me) a few minutes of reimplementing it for something else.

Please leave a comment if you notice anything broken, and I'd appreciate any feedback, as always.

Subscribe to my mailing list

Did you like what you just read and want to be notified when I post more? Subscribe to my mailing list to get an email whenever I post.
